

Heart Disease Prediction - AI Project

Prepared for:

Dr.Verena Nashaat

Prepared by:

NeuralCore Team

Prepared by:

Mohammed Hamdy Abdulrahim	(20241700715)
Ahmed Yasser Abd elmaqsoud	(20241700087)
Maher Sayed Abdelshahid	(20241700679)
Mohammed Al-Sayed Abdullah	(20241700702)
Mahmoud Abdelmegeed Abdelkader	(20241700780)
Ashraf Mohamed Abd-Elwahab	(20241700123)

Date of Submission: 5-9-2026

Table of contents:

1	Introduction to Heart Disease Prediction - AI Project	1
2	Dataset Overview	2
3	Project Flow	5
2.1	Preprocessing	5
2.1.1	Dropping Non-Useful Columns	5
2.1.2	Handling Missing Values	5
2.1.3	Data Type Casting	5
2.1.4	Dropping Duplicates	6
2.1.5	Handling Outliers	6
2.2	Features & Target Separation	6
2.3	Encoding	6
2.4	Scaling	7
2.4	Correlation	7
2.5	Model Training	8
2.2.1	Logistic Regression Model	8
2.2.2	SVM Model	9
2.2.3	Decision Tree Model	11
2.2.4	KNN Model	13
2.2.5	Random Forest.....	15
4	Conclusion	16

Introduction to Heart Disease Prediction - AI Project

Heart disease is one of the top causes of death worldwide, causing an estimated **16.8 deaths** each year. Diagnosing methods take time and require numerous tests. With the help of Artificial intelligence and Machine learning, detecting heart disease is now more efficient and easier to identify at the early stages and the potential risks of heart disease.

This project introduces a system for detecting heart disease using algorithms to predict whether a patient is likely to have it based on medical attributes such as age, cholesterol levels, fasting blood sugar, electrocardiogram, maximum heart rate, exercise angina, ST depression, number of vessels fluoroscopy, and smoking status.

Our main goal is to reduce human error, improve diagnostic accuracy, and efficiency. The model is trained using a medical dataset and a variety of AI training methods to ensure efficiency and performance.

Dataset Overview

The dataset used in this project consists of two separate CSV files:

- Training data : [Link](#)
- Testing data : [Link](#)

Dataset Size

Training: 224 rows, Testing: 56 rows

Features (17 Columns)

The dataset contains the following columns:

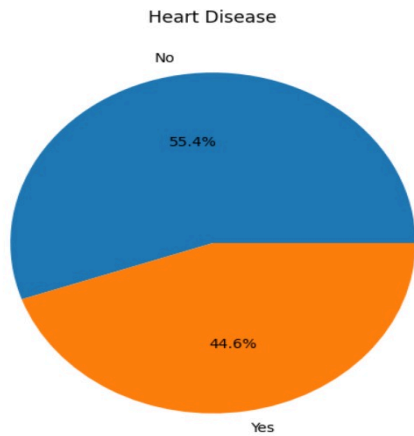
id, Age, Gender, Chest pain type, BP, Cholesterol, FBS over 120, EKG results, Max HR, Exercise angina, ST depression, Slope of ST, Number of vessels fluoro, Thallium, work_type, smoking_status, Heart Disease

Target Column

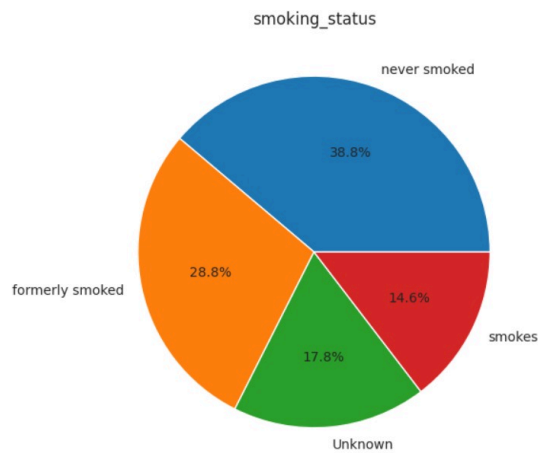
- Heart Disease : The target variable (binary classification)

Training Dataset Summary

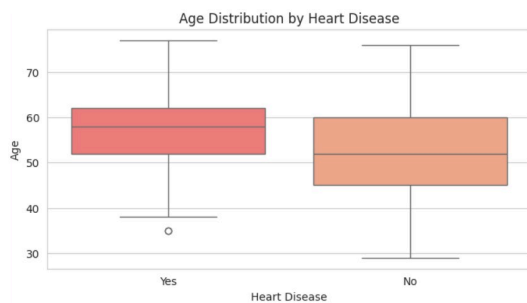
	Name	Data_Type	Top_10_Unique_Values	Nunique_Values	Nulls	Percent_of_Nulls	Duplicates
0	id	int64	[134, 133, 132, 131, 130, 129, 128, 136, 135, 10]	215	0	0.000000	9
1	Age	float64	[54.0, 58.0, 57.0, 59.0, 60.0, 42.0, 51.0, 64....	41	4	1.785714	9
2	Gender	object	[Female, Male]	2	6	2.678571	9
3	Chest pain type	int64	[4, 3, 2, 1]	4	0	0.000000	9
4	BP	int64	[120, 140, 130, 110, 150, 125, 112, 160, 128, ...	45	0	0.000000	9
5	Cholesterol	int64	[269, 234, 226, 211, 177, 230, 197, 246, 233, ...	122	0	0.000000	9
6	FBS over 120	int64	[0, 1]	2	0	0.000000	9
7	EKG results	int64	[2, 0, 1]	3	0	0.000000	9
8	Max HR	int64	[160, 163, 125, 162, 165, 178, 169, 152, 173, ...	81	0	0.000000	9
9	Exercise angina	int64	[0, 1]	2	0	0.000000	9
10	ST depression	float64	[0.0, 0.8, 1.4, 1.2, 0.2, 0.6, 1.6, 1.0, 0.1, ...	37	0	0.000000	9
11	Slope of ST	int64	[1, 2, 3]	3	0	0.000000	9
12	Number of vessels fluoro	int64	[0, 1, 2, 3]	4	0	0.000000	9
13	Thallium	int64	[3, 7, 6]	3	0	0.000000	9
14	work_type	object	[Private, Self-employed, Govt_job, children]	4	7	3.125000	9
15	smoking_status	object	[never smoked, formerly smoked, Unknown, smokes]	4	5	2.232143	9
16	Heart Disease	object	[No, Yes]	2	0	0.000000	9



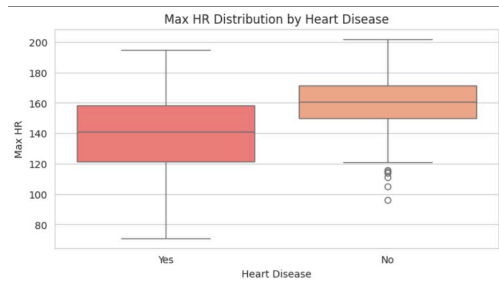
The target distribution is fairly balanced (55.4% No vs 44.6% Yes), so no class balancing was required.



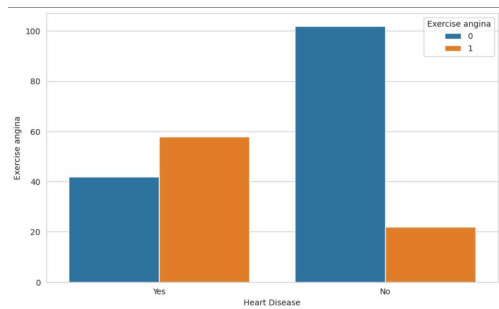
With 17.8% 'Unknown' values and 2.23% missing values (~20% unreliable data), the smoking_status column was a candidate for removal



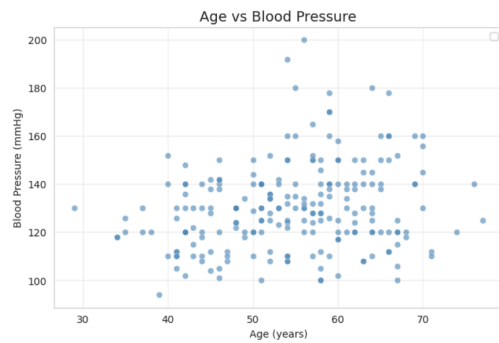
The analysis shows that older age is associated with higher likelihood of heart disease.



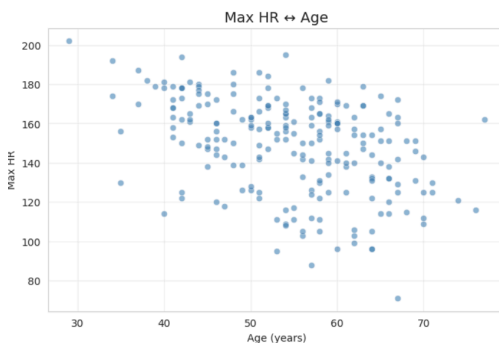
Patients without heart disease show higher Max HR values.



Most patients without heart disease (~ 80%) have no Exercise angina



Older age tends to be associated with higher blood pressure.



Older patients tend to have slower maximum heart rates.

Project Flow:

Preprocessing:

Dropping Non-Useful Columns

The following columns were dropped from both the training and testing datasets to ensure feature consistency.

- id : Dropped because it is non-predictive and irrelevant to the target.
- Smoking_status : Dropped after experimentation. It contained 17.8% Unknown values and 2.2% NaN. Imputing with mode gave lower overall accuracy. Removing the column entirely improved model performance.

Handling Missing Values

Missing values represented less than 4% of total rows in both train and test sets. Rows containing null values were dropped entirely (listwise deletion).

Features that contained null values: Age, Gender, work_type.

After removal:

- Train data: 198 rows
- Test data: 51 rows

Verified that the remaining dataset size was still sufficient for reliable model train and evaluation.

Data Type Casting

Data Type Casting (applied identically to both training and testing datasets)

- Age : Casted from float64 to int64 (all values were whole numbers)
- Categorical columns (containing numeric codes that represent categories) were casted from int64 to object to prevent models from interpreting them as ordinal numerical values: Chest pain type, FBS over 120, EKG results, Exercise angina, Slope of ST, Number of vessels fluro, Thallium

Dropping Duplicates

Duplicate rows were identified and removed from both training and testing datasets to avoid data leakage and bias.

Handling Outliers

Handling Outliers using IQR (Interquartile Range) – applied to both train and test

Calculated Q1, Q3, and IQR from training data only

Defined bounds:

- Lower bound = $Q1 - 1.5 \times IQR$
- Upper bound = $Q3 + 1.5 \times IQR$

Capping :

- Any value < lower bound → set to lower bound
- Any value > upper bound → set to upper bound

Same bounds applied to both train and test to prevent data leakage

Features & Target Separation

The target column Heart Disease was separated from features in both datasets.

Encoding

Encoding (One-Hot Encoding)

- Identified categorical columns with object dtype
- Applied `pd.get_dummies()` to both train and test
- Aligned test columns with train using `reindex(columns=x_train.columns, fill_value=0)`
-

Target Mapping

- `y_train : {'No': 0, 'Yes': 1}`
- `y_test : {'No': 0, 'Yes': 1}`

Scaling

Scaling (Min-Max Normalization)

- Calculated min and max from training data only
- Applied to numerical columns: $(x - \min) / (\max - \min)$
- Same transformation applied to test using train-derived values

Correlation

Two statistical tests were used to evaluate feature importance:

- Numerical features → ANOVA F-test
- Categorical features → Chi-square test

Weak Features (High P-Value > 0.05):

Cholesterol, FBS over 120, work_type, smoking_status

Each Weak feature was tested by removing it and comparing model accuracy.

Result:

- Removing smoking_status → **improved** overall accuracy
- Removing each of Cholesterol, FBS over 120, work_type → **did not** improve accuracy

Model Training:

Logistic Regression Model

Logistic Regression is a supervised machine learning model used for classification tasks. It predicts the probability that an input belongs to a certain class, typically using a sigmoid function to map values between 0 and 1. Despite its name, it is used for classification rather than regression. Logistic regression is simple, efficient, and widely used for binary classification problems like spam detection or disease prediction.

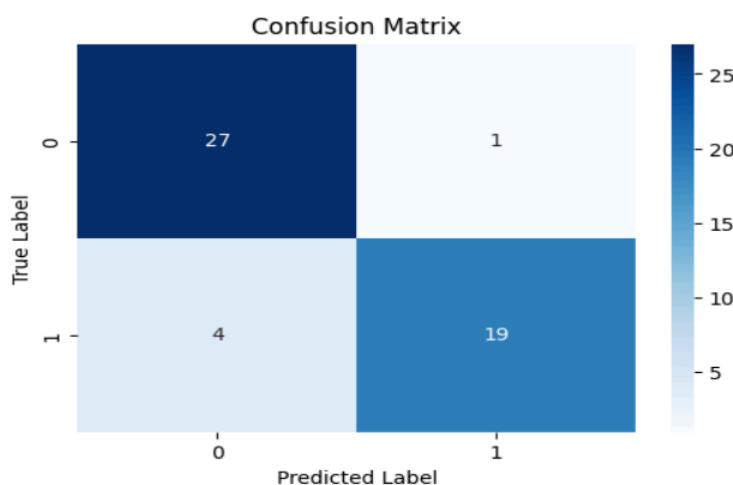
Techniques Used:

- Feature scaling
- Logistic Regression
- Hyperparameter tuning (GridSearchCV)
- Cross-validation
- Model evaluation metrics (Accuracy, F1-score, ROC-AUC)

Best Parameters: {'C': 1, 'penalty': 'l2', 'solver': 'liblinear'}

The model achieved a test accuracy of 90.19% and a train accuracy of 91.99% , which indicates good performance.

The dataset consists of X samples and Y features. The target variable represents whether a patient has heart disease (1) or not (0).



Support Vector Machine (SVM) Model

Support Vector Machine (SVM) is a supervised Machine Learning algorithm used for classification problems.

. It works by finding the optimal hyperplane that best separates different classes in the dataset. The main idea of SVM is to maximize the margin between the closest data points of different classes, which are called support vectors. A larger margin generally leads to better generalization on unseen data.

In this implementation, different kernel functions were tested to improve model performance. A kernel is used to transform the data into a higher-dimensional space to make it easier to separate non-linearly separable data.

The kernels used in this model were:

- Linear Kernel: used when data is linearly separable
- RBF (Radial Basis Function) Kernel: used for non-linear classification

The model was trained using different values of the hyperparameter **C**. The parameter C controls the trade-off between maximizing the margin and minimizing classification error. A smaller C allows a wider margin with more misclassifications, while a larger C focuses on classifying training data correctly but may lead to overfitting.

The tested hyperparameters were:

- C values: 0.1, 1, 10
- Kernel types: linear, rbf

The model was trained multiple times using all combinations of these parameters to find the best configuration that gives the highest accuracy.

For each combination, the model was evaluated using:

- Accuracy Score
- Classification Report

The classification report included:

- Precision
- Recall
- F1-Score

These metrics were used to measure the performance of the model in predicting different classes.

The best hyperparameters were selected based on the highest test accuracy achieved during experimentation. The final best model is the one with the optimal combination of kernel and C value.

The SVM model achieved strong classification performance for the heart disease prediction problem. Testing different kernels and tuning the C parameter helped improve model accuracy and ensure better generalization on unseen data

Decision Tree Model

Decision Tree is a supervised Machine Learning algorithm used for classification problems. It works by splitting the dataset into multiple nodes based on the feature that provides the highest Information Gain.

The tree starts with a Root Node, then branches into internal nodes, and finally reaches Leaf Nodes that represent the final prediction.

Decision Trees are simple to understand, easy to visualize, and effective for classification tasks.

The model was trained using the entropy criterion. Entropy measures the impurity or randomness in the data. Lower entropy means better separation between classes.

The Decision Tree selects the best feature for splitting using Information Gain.

Information Gain equation:

$$IG = \text{Entropy}(\text{Parent}) - \sum (N_i / N) \times \text{Entropy}(\text{Child}_i)$$

A feature with higher Information Gain is considered a better feature for splitting.

Several hyperparameters were optimized using GridSearchCV to improve model performance and reduce overfitting. The tuned hyperparameters included:

- max_depth
- min_samples_split
- min_samples_leaf
- criterion = entropy

5-Fold Cross Validation was used during training to select the best parameter combination automatically.

The model performance was evaluated using:

- Train Accuracy
- Test Accuracy
- Cross Validation Accuracy
- Classification Report

The classification report included:

- Precision
- Recall
- F1-Score

These metrics were used to measure the prediction performance of the model.

The final Decision Tree was visualized using `plot_tree()` to explain how the model makes decisions.

Information Gain was also calculated manually for each node to better understand the splitting process inside the tree.

The Decision Tree model achieved good classification performance for predicting heart disease cases. Using entropy, Information Gain, and hyperparameter tuning with GridSearchCV helped improve the model accuracy and reduce overfitting.

KNN Model

The K-Nearest Neighbors is a supervised learning algorithm based on the assumption that samples close to each other are likely to belong to the same class. The model stores the training dataset and computes the distance between the new sample and all points in the training data. It then selects the K nearest neighbors, where K is a hyperparameter representing the number of neighboring samples used for classification. The value of K must be carefully chosen. A very small K value may lead to overfitting because the model becomes sensitive to noise in the dataset while a very large K value may lead to underfitting because the model memorizes the data. In classification tasks, KNN uses majority voting among the nearest neighbors to determine the predicted class, while in regression tasks it uses averaging.

To improve model performance and select the optimal K value, the following techniques were used:

- Hyperparameter tuning
- Elbow plot

Different K values were tested, and the model accuracy was calculated for each value of k. The elbow plot was then used to visualize the relationship between K values and accuracy in order to identify the optimal K value for the model.

As shown in the following elbow plot with best accuracy reached = 88% :

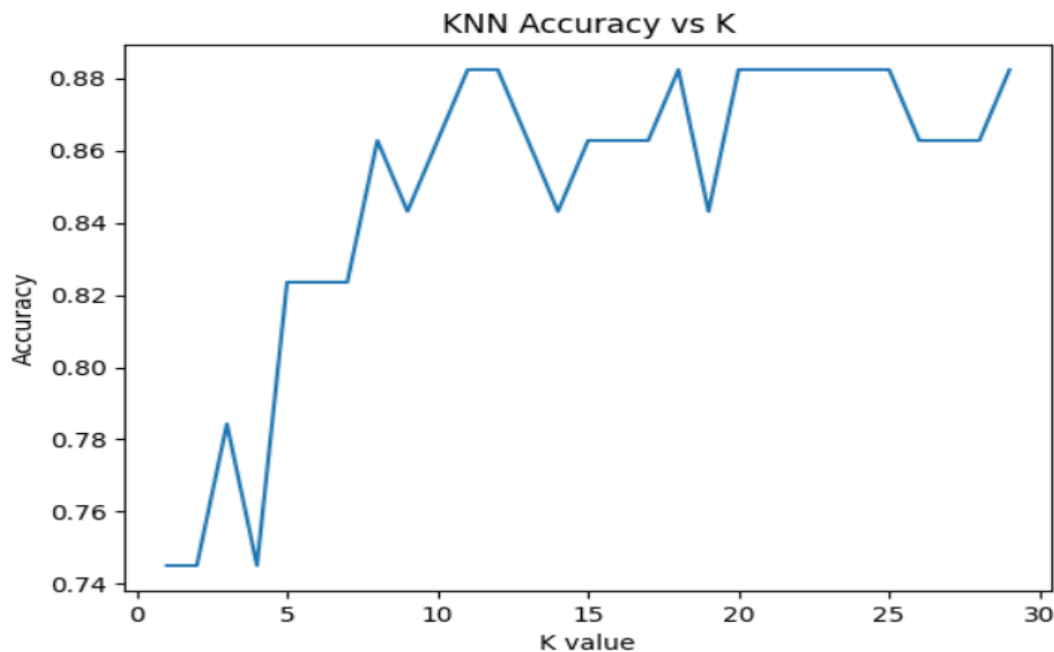


Figure1 depicts the Negative Impact of Social Media ,which is discussed in the previous five research papers.

Each of the studies reviewed in this report points to a shared concern: that our relationship with digital technology is changing the way we relate to others and ourselves. Turkle demonstrates unequivocally that constant connectivity does not eliminate feelings of solitude; rather, many emerge as more distracted, distant from others, and especially so when face-to-face relations are replaced by frequent texting or quick online exchanges. This appears rather clearly among younger groups, who read delayed responses as a form of conflict and become increasingly dependent on digital reassurance.

Quantitative evidence reinforces this idea. Primack and colleagues discovered that heavy use of social media is associated with greater perceived social isolation, implying that computer-mediated interactions may be displacing, rather than complementing, in-person interactions. Research on self-image provides another layer. Numerous recent studies demonstrate how perpetual exposure to filtered images and idealized lives perpetuates unhealthy comparisons, lowering confidence and building pressure on young users. Nevertheless, other findings, such as those from Goldfield's trial, show that the negative effects may not be permanent.

More recent work on algorithms adds an important dimension. Hu's research indicates how engagement-based systems push emotionally charged or harmful content, affecting mental well-being and addictive patterns. Large-scale analyses of bots show that a high number of online activities are automated and shape conversations in ways that users might not notice. Taken together, these studies crystallize an imperative for more transparent and humane digital environments—one supportive of healthier communication and personal well-being.

Random Forest Model

Random Forest is a supervised machine learning algorithm used for classification and regression tasks . It works by building multiple decision trees during training and combining their predictions to produce more accurate and stable results . For classification problems, the model predicts the final output based on the majority vote of all trees. Random Forest is widely used because it can handle complex datasets , reduce overfitting , and provide high prediction accuracy in applications such as medical diagnosis and disease prediction.

Techniques Used:

Feature scaling

Random Forest Classifier

Hyperparameter tuning (GridSearchCV)

Cross-validation

Model evaluation metrics (Accuracy, F1-score, ROC-AUC)

Best Parameters:

```
{'class_weight': None, 'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 3, 'min_samples_split': 4, 'n_estimators': 250}
```

The model achieved a train accuracy of 92.56% and a test accuracy of 89.09%, which indicates good performance and strong generalization on unseen data.

The dataset consists of X samples and Y features. The target variable represents whether a patient has heart disease (1) or not (0).

Conclusion

This project set out to answer a simple question can AI actually help catch heart disease before it's too late? Based on the results, the answer is a promising yes. By looking at everyday health indicators things such as age, cholesterol levels, fasting blood sugar, electrocardiogram, maximum heart rate, exercise angina, ST depression, number of vessels fluoroscopy, and smoking status, the model was able to identify warning signs that might otherwise go unnoticed until it's too late. That kind of early visibility matters. A diagnosis made weeks or months earlier can completely change a patient's path forward.

The goal was never to replace doctors. If anything, it's the opposite — giving healthcare professionals a smarter, faster tool to support their decisions, especially in settings where time and resources are stretched thin.

Of course, no first attempt is perfect. The model has real limitations, and there's honest work still to be done: more diverse data, broader testing, and continued refinement. But the foundation is solid, and the direction is clear.

At its core, this project is a small piece of a much bigger shift happening in medicine, one where technology doesn't just assist human judgment, but actively helps protect human lives. That's a goal worth building toward.

Models Accuracy & Best Hyperparameters

Model	Best Accuracy	Best Hyper Parameters
Logistic Regression	90.20%	'C': 1, 'penalty': 'l2', 'solver': 'liblinear'
SVM	90.20%	'kernel': 'linear', 'C': 10
Decision Tree	84.31%	'criterion': 'entropy', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2
KNN	88.23%	K : 11
Random Forest	86.27%	'Class_weight': 'balanced', 'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 3, 'min_samples_split': 4, 'n_estimators': 100